

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ № 5

"Изучение основ работы с менеджером зависимостей npm"

Выполнил:
Демьянов Фёдор

Группа: J3214

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Освоить базовые принципы построения одностраничных веб-приложений (SPA) на базе фреймворка Vue 3. Изучить работу с компонентным подходом, настроить клиентскую маршрутизацию через Vue Router, организовать централизованное управление состоянием с помощью Pinia, а также реализовать асинхронное взаимодействие с локальным REST API через библиотеку Axios с применением принципа внедрения зависимостей.

2. Теоретическая база и архитектура решения

Для создания приложения использован стек современных инструментов веб-разработки:

- **Vue 3:** реактивный прогрессивный фреймворк для создания пользовательских интерфейсов. В компонентах использован Options API для демонстрации работы с методами, вычисляемыми свойствами и хуками жизненного цикла.
- **Vite:** быстрый инструмент сборки и локальный сервер разработки с поддержкой горячей замены модулей (HMR).
- **Vue Router:** официальный маршрутизатор для Vue, обеспечивающий переход между экранами без перезагрузки всей страницы браузера. В проекте настроена ленивая загрузка компонентов (Lazy Loading).
- **Pinia и pinia-persists:** официальная библиотека управления состоянием, заменяющая устаревший Vuex. Плагин **pinia-persists** синхронизирует состояние заметок с **localStorage**, предотвращая потерю данных при обновлении страницы.
- **Axios и паттерн Dependency Injection (DI):** HTTP-клиент не вызывается напрямую внутри компонентов. Для работы с эндпоинтами создан отдельный класс **NotesApi**, который принимает экземпляр Axios через свой конструктор. Роль IoC-контейнера выполняет файл **index.js**, экспортирующий уже настроенный модуль.
- **Bootstrap 5:** библиотека адаптивных стилей для оформления карточек, кнопок и формы ввода.
- **json-server:** локальный сервер, предоставляющий REST API на основе файла **db.json**.

3. Практическая реализация

3.1. Инициализация проекта и настройка окружения

Развертывание стартового каркаса приложения выполнено через утилиту **create-vue** (команда **npm init vue@latest**). При интерактивной настройке в проект были включены модули Vue Router и Pinia.

В файле **package.json** скорректирован блок пользовательских скриптов:

- команда `start` настроена на запуск сборщика Vite на фиксированном порту 8080 (`vite --port 8080`);
- добавлен скрипт `server` для фонового запуска базы данных (`json-server --watch db.json --port 3000`).

В корне проекта создан файл базы данных `db.json`, содержащий массив заметок с полями `id`, `name`, `text` и `userId`.

3.2. Слой API и внедрение зависимостей

Сетевой слой вынесен в директорию `src/api/` и разделен на три изолированных файла:

1. `instance.js`: создает базовый экземпляр Axios с адресом локального сервера (`http://localhost:3000`).
2. `notes.js`: содержит класс `NotesApi` с методами `getAll()` для получения заметок и `createNote(data)` для отправки новой записи методом POST. Экземпляр HTTP-клиента передается через конструктор:

```
JavaScript
class NotesApi {
  constructor(instance) {
    this.API = instance
  }
  getAll = async () => this.API({ url: '/notes' })
  createNote = async (data) => this.API({
    method: 'POST',
    url: '/notes',
    data,
    headers: { 'Content-Type': 'application/json' }
  })
}
export default NotesApi
```

3. `index.js`: связывает класс с экземпляром Axios и отдает готовый объект `notesApi` наружу.

3.3. Централизованное хранилище Pinia

В директории `src/stores/` создано хранилище `notes.js`:

- **State:** реактивный массив `notes`, в котором сохраняются полученные с сервера записи.
- **Actions:** метод `loadNotes()` запрашивает данные через `notesApi.getAll()` и записывает их в стейт; метод `createNote(data)` выполняет отправку на сервер и дополняет локальный

массив созданной заметкой. В `src/stores/index.js` подключен плагин персистентности, гарантирующий сохранение данных при перезагрузках страницы.

3.4. Маршрутизация и структура компонентов

В приложении реализовано строгое разделение ответственности между компонентами:

- `src/layouts/BaseLayout.vue`: макет-обертка, задающий ширину контейнера через классы Bootstrap. Для вывода динамического содержимого страницы внутри макета используется тег `<slot />`.
- `src/components/NoteCard.vue`: дочерний переиспользуемый компонент. Принимает входные параметры `name` и `text` через секцию `props`, отображая их внутри карточки Bootstrap.
- `src/views/NotesPage.vue`: главное представление. Связано с хранилищем через вспомогательные функции `mapState` (получение массива `notes`) и `mapActions` (вызов методов `loadNotes` и `createNote`).
- `src/router/index.js`: настраивает переход по корневому пути `/` на страницу `NotesPage` с использованием синтаксиса динамического импорта.

В форме добавления заметки применены ключевые директивы Vue:

- `v-model`: обеспечивает двустороннюю привязку введенных пользователем строк к локальным переменным `form.name` и `form.text`;
- `@submit.prevent="createCard"`: перехватывает стандартную отправку HTML-формы, предотвращая перезагрузку страницы, и вызывает метод добавления заметки;
- `v-for="note in notes" :key="note.id"`: циклически рендерит компоненты `NoteCard` на основе данных из хранилища `Pinia`.

Хук жизненного цикла `mounted()` автоматически запускает метод `loadNotes()` сразу после монтирования представления в дерево DOM, благодаря чему пользователь видит сохраненные заметки без дополнительных действий.

4. Результаты тестирования

Для подтверждения корректной работы сервиса проведена проверка ключевых сценариев:

1. **Запуск сервисов:** сервер базы данных `json-server` успешно запустился на порту 3000, а сервер разработки Vite открыл приложение на порту 8080.

```
PS C:\Users\GeekyMars\Desktop\Homework\front\ITMO-AIT-Frontend-2026\33214\Демьянов Фёдор\homeworks\hw5> npm start

> hw5@0.0.0 start
> vite --port 8080

VITE v8.3.0 ready in 729 ms

→ Local:   http://localhost:8080/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:8080/__devtools__/ as a separate window
→ Vue DevTools: Press Alt(⇧)+Shift(⇧)+D in App to toggle the Vue DevTools
→ press h + enter to show help

PS C:\Users\GeekyMars\Desktop\Homework\front\ITMO-AIT-Frontend-2026\33214\Демьянов Фёдор\homeworks\hw5> npm run server

> hw5@0.0.0 server
> json-server --watch db.json --port 3000

\{^_^}/ hi!

Loading db.json
Done

Resources
http://localhost:3000/notes

Home
http://localhost:3000

Type s + enter at any time to create a snapshot of the database
Watching...
```

Рисунок 1: Окна терминала с запущенными командами `npm run server` и `npm start`

2. **Файловая структура:** все модули разнесены по соответствующим директориям в соответствии с принципами чистой архитектуры.

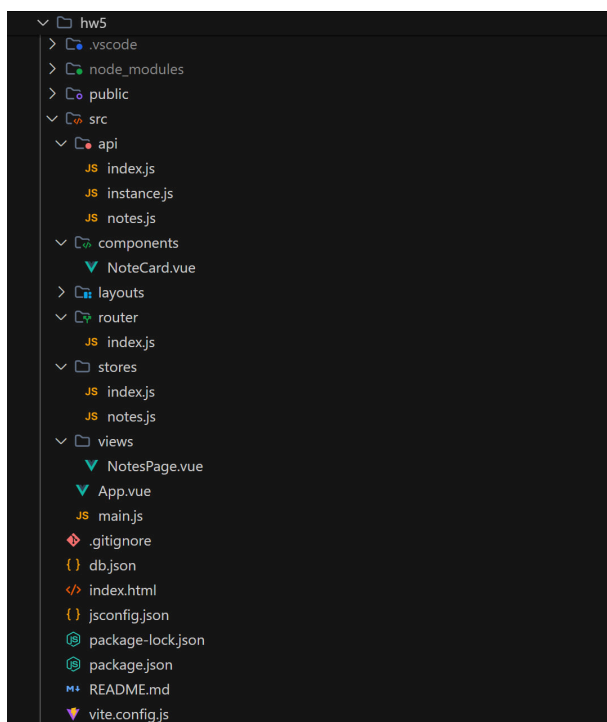
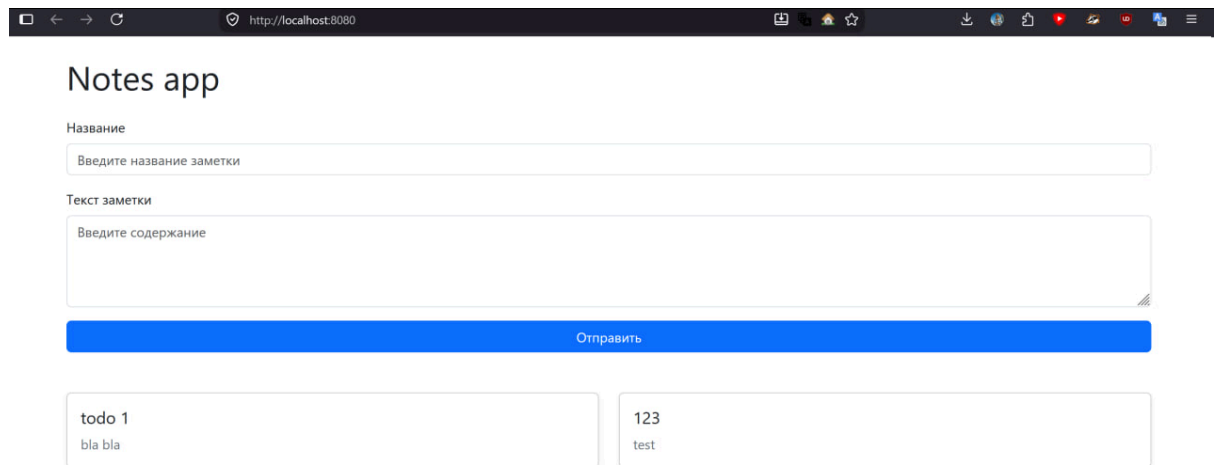


Рисунок 2: Дерево каталогов проекта в проводнике VS Code

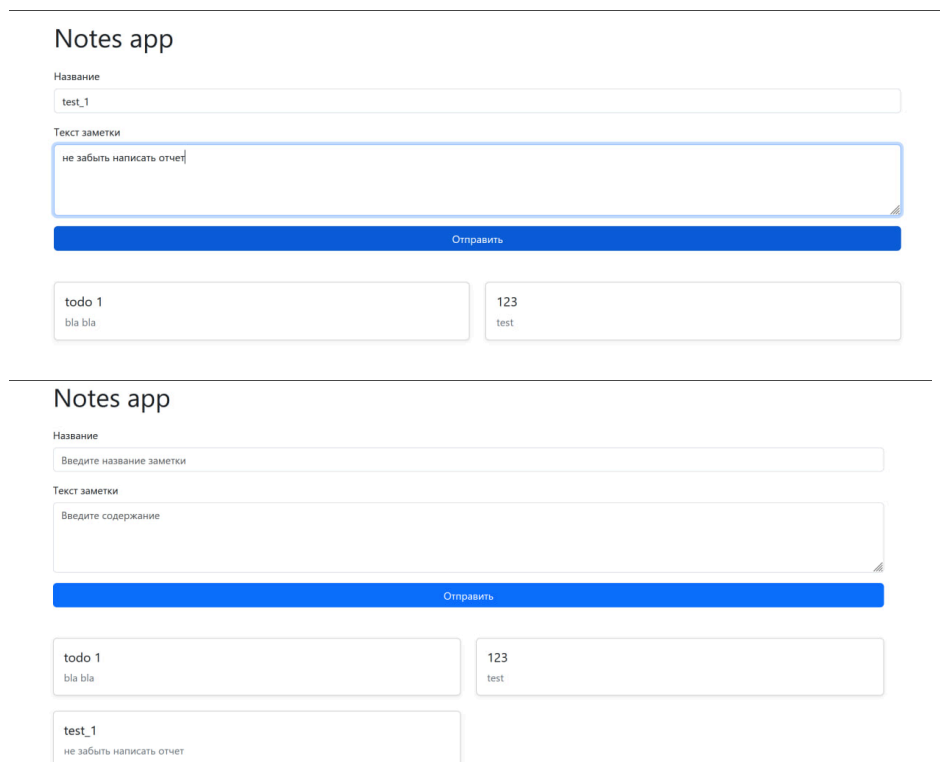
3. **Первичная отрисовка:** при загрузке страницы хук `mounted()` обратился к серверу и вывел две тестовые заметки из файла `db.json`.



The screenshot shows a web browser at `http://localhost:8080` displaying the 'Notes app'. The form has two input fields: 'Название' (Name) with placeholder 'Введите название заметки' and 'Текст заметки' (Text) with placeholder 'Введите содержание'. Below the form is a blue 'Отправить' (Send) button. At the bottom, there are two note cards. The first card has a title 'todo 1' and content 'bla bla'. The second card has a title '123' and content 'test'.

Рисунок 3: Стартовая страница `http://localhost:8080/` с начальными карточками

4. **Добавление новых заметок:** в форму были введены данные заметок `test_1` и `test_2`. После нажатия кнопки «Отправить» форма очистилась, а созданные карточки мгновенно отобразились в конце сетки без перезагрузки экрана.



The first screenshot shows the 'Notes app' form with the 'Название' field containing 'test_1' and the 'Текст заметки' field containing 'не забыть написать отчет'. The 'Отправить' button is highlighted. The second screenshot shows the same app after submission. The form is now empty, and a new note card has been added to the bottom of the list. This new card has a title 'test_1' and content 'не забыть написать отчет'. The existing cards 'todo 1' and '123' remain in their original positions.

Рисунок 4: Заполнение полей формы

5. **Работа сети и хранилища:** на панели инструментов разработчика (вкладка Network) зафиксирован успешный HTTP-запрос `GET /notes` с кодом ответа `200 OK`. В консоли

браузера подтверждена корректная регистрация хранилища Pinia ('notes' store installed).

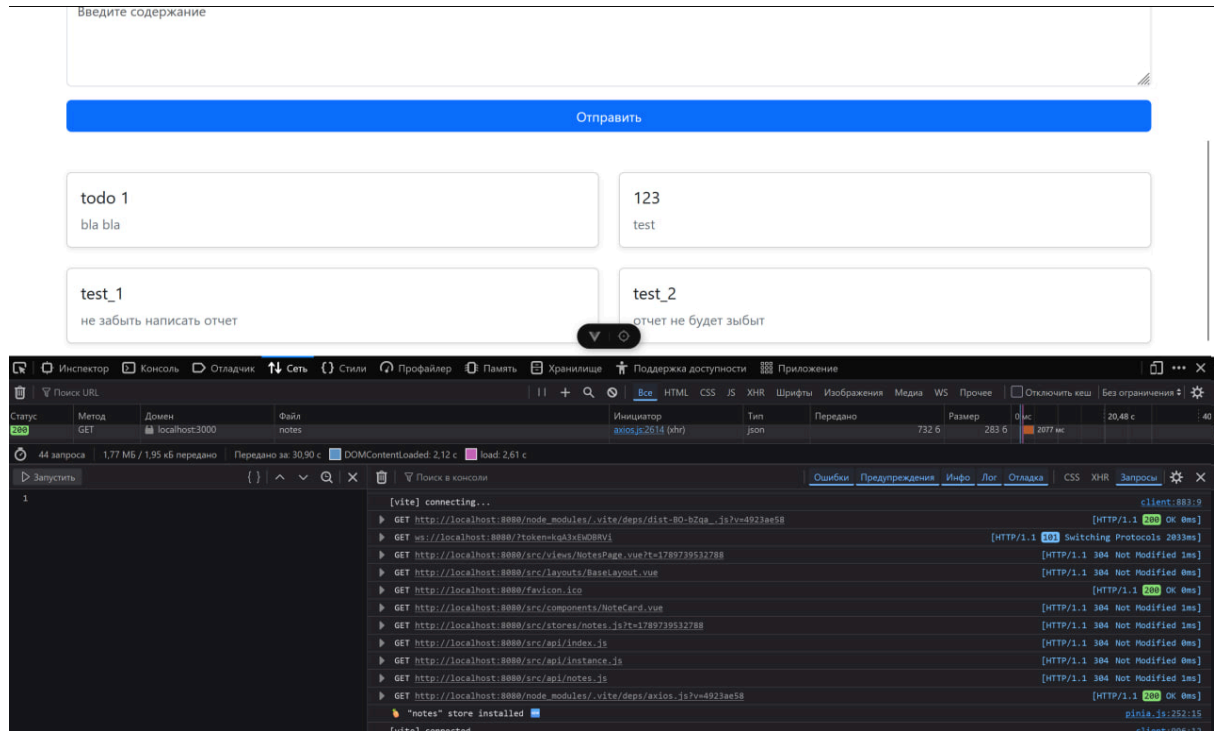


Рисунок 5: Интерфейс приложения с новыми заметками, открытой вкладкой Network и логами консоли

6. **Синхронизация с базой данных:** файл `db.json` автоматически обновился на диске, зафиксировав созданные записи.



Рисунок 6: файл `db.json` с новыми объектами

5. Заключение

В рамках выполнения домашней работы № 5 были изучены базовые возможности фреймворка Vue 3 и сопутствующей экосистемы.

Главный вывод из работы заключается в том, насколько компонентный подход и реактивная система упрощают разработку интерфейсов. В отличие от классического JavaScript, где приходилось вручную находить узлы в DOM-дереве и собирать строки HTML через `innerHTML`, во Vue достаточно описать шаблон и связать его с данными через `v-model` и `v-for`. Изменение стейта само вызывает точечную перерисовку нужных элементов на странице.

Вынос сетевых запросов в отдельный слой с использованием внедрения зависимостей сделал код масштабируемым и независимым от конкретных компонентов, а связка Vue Router и Pinia позволила получить полноценный каркас SPA-приложения, готовый к реализации более сложных проектов.

В контексте развития сервиса «NovaTransit» эта домашняя работа стала практическим полигоном перед выполнением третьей лабораторной работы. Во второй лабе приходилось вручную управлять DOM-деревом в `catalog.js`, собирать карточки через `innerHTML` и писать длинные императивные обработчики для пересчета стоимости в `tour.js`. Теперь логика миграции проекта в архитектуру SPA стала понятной и прозрачной: расчет дополнительных опций полета и интерактивную смену планет на главном экране гораздо чище реализовать на реактивном стейте и директивах вроде `v-model`, переходы между терминалами авторизации, каталогом и личным кабинетом переложить на Vue Router, а сетевое взаимодействие с базой `db.json` и шлюзом NASA унифицировать в едином модульном слое Axios.